

Azure AI Foundry (DeepSeek-V3.2) in VS Code Copilot via LiteLLM

Route GitHub Copilot Chat (BYOK) through a LiteLLM proxy to an Azure AI Foundry model — fixing 429 rate limits and the `copilot-utility-small` BYOK error.

VS Code extension `johnny-zhao.oai-compatible-copilot` v0.4.2 · LiteLLM v1.83.14 · Azure AI Foundry (eastus2) · DeepSeek-V3.2

What you get: GitHub Copilot Chat in VS Code uses a DeepSeek-V3.2 model hosted on Azure AI Foundry — with the LiteLLM proxy in the middle to absorb rate-limit bursts (retry + backoff + cooldown) and a properly configured BYOK utility model so Copilot's background tasks don't error.

0 Architecture

```
VS Code + Copilot Chat
| extension: oai-compatible-copilot
| model: deepseek-v3.2-foundry
▼
LiteLLM Proxy (http://YOUR-SERVER:4000)
| model group: deepseek-v3.2-foundry
| · retries 429 with exponential backoff
| · cools down the deployment on repeated fails
▼
Azure AI Foundry (DeepSeek-V3.2, eastus2)
https://YOUR-RESOURCE.services.ai.azure.com/openai/v1

Utility model (background tasks):
Copilot Chat — chat.utilitySmallModel —> oaicopilot/glm-5.1 —> Huawei MaaS
```

1 Prerequisites

- A running **LiteLLM proxy** (e.g. `http://YOUR-SERVER:4000`) with access to edit its `litellm_config.yaml` and `.env`, and a **master/virtual key** for it.
- An **Azure AI Foundry** account with a **DeepSeek-V3.2** deployment (serverless) in a region (e.g. `eastus2`), and its **API key**.
- VS Code with the **OAI Compatible Provider for Copilot** extension installed (`johnny-zhao.oai-compatible-copilot`).
- GitHub Copilot Chat enabled in VS Code.
- A fast, high-quota model reachable from LiteLLM to serve as the **utility model** (e.g. GLM-5.1 on Huawei MaaS). Any OpenAI-compatible model with tool-calling works.

2 The two problems this solves

Problem 1 — 429 RateLimitReached: Copilot Chat fires several concurrent requests (main reply + title + follow-up suggestions) in a burst. Pointing the extension **directly** at Foundry exceeds DeepSeek-V3.2's per-deployment rate limit → [429 RateLimitReached](#).

Problem 2 — "No utility model is configured for 'copilot-utility-small'": When the main agent model is BYOK, Copilot Chat needs a **utility model** for background tasks. It defaults to GitHub's hosted [copilot-utility-small](#), which isn't available in BYOK mode → error.

3 Add the Foundry model to LiteLLM

Edit `litellm_config.yaml` on the LiteLLM server. Add a model group that points at your Foundry endpoint. The `model` is the Foundry **deployment name**; `api_base` is the Foundry OpenAI-compatible endpoint (without `/chat/completions`).

```
# — Azure AI Foundry: DeepSeek-V3.2 (eastus2, serverless) —
model_list:
  - model_name: deepseek-v3.2-foundry
    litellm_params:
      model: openai/DeepSeek-V3.2
      api_base: https://YOUR-RESOURCE.services.ai.azure.com/openai/v1
      api_key: os.environ/AZURE_FOUNDRY_DEEPSEEK_KEY
      tpm: 50000
      rpm: 30
    model_info:
      max_tokens: 160000
      max_input_tokens: 128000
      max_output_tokens: 32000
```

Why this fixes the 429: LiteLLM sits between the extension and Foundry. When Foundry returns 429, LiteLLM **retries with exponential backoff** (`num_retries` , `cooldown_time`) instead of surfacing the error. Set `rpm` / `tpm` at or below your Foundry quota so LiteLLM throttles outbound calls.

3.1 — Add the Foundry API key to `.env`

```
# Azure AI Foundry - DeepSeek-V3.2
AZURE_FOUNDRY_DEEPSEEK_KEY=YOUR-FOUNDRY-API-KEY
```

3.2 — Restart LiteLLM

```
cd /home/YOUR-LITELLM-DIR
docker compose up -d litellm          # recreate to load the new env var
# (or: docker restart litellm_proxy if only the yaml changed)
```

3.3 — Verify

```
curl -s http://127.0.0.1:4000/v1/chat/completions \
-H 'content-type: application/json' \
-H 'Authorization: Bearer YOUR-LITELLM-KEY' \
-d '{"model":"deepseek-v3.2-foundry","messages":[{"role":"user","content":"hi"}],"max_tokens":10}'
```

Expected: HTTP 200 with a normal completion.

4 Configure the VS Code extension

The extension is configured in VS Code `settings.json` (Windows: `%APPDATA%\Code\User\settings.json`). Add a model entry that routes through LiteLLM instead of directly to Foundry.

4.1 — Add the model to `oaicopilot.models`

```
"oaicopilot.models": [
  {
    "id": "deepseek-v3.2-foundry",
    "displayName": "DeepSeek-V3.2 (Foundry via LiteLLM)",
    "owned_by": "litellm",
    "baseUrl": "http://YOUR-SERVER:4000/v1",
    "apiMode": "openai",
    "context_length": 128000,
    "max_tokens": 32000,
    "headers": {
      "Authorization": "Bearer YOUR-LITELLM-KEY"
    }
  }
]
```

Field	Meaning
<code>id</code>	Must match the LiteLLM <code>model_name</code> exactly. This is sent as the <code>model</code> field to LiteLLM.
<code>owned_by</code>	Provider label; used to group API keys. Any string works.
<code>baseUrl</code>	LiteLLM's OpenAI-compatible base URL.
<code>apiMode</code>	<code>"openai"</code> = Chat Completions (<code>/chat/completions</code>). Use <code>"openai-responses"</code> for the Responses API.
<code>headers.Authorization</code>	Embeds the LiteLLM key. Custom headers override the default Bearer header. This avoids needing the command-palette key step.

Key resolution note: When a model has a per-model `baseUrl`, the extension looks for a provider-specific key (`oaicopilot.apiKey.<owned_by>`) in secret storage. If not found, it may prompt you once. You can either enter the LiteLLM key, press **Esc** (the custom `headers.Authorization` still authenticates), or pre-set it via **OAI Copilot: Set OAI Compatible Multi-Provider API Key**.

5 Fix the BYOK utility model error

This is the critical step. Without it, selecting the BYOK model as the main agent produces: *"No utility model is configured for 'copilot-utility-small' while the selected main agent model is BYOK."*

GitHub Copilot Chat has two core VS Code settings for the utility model. Add them to `settings.json`:

```
"chat.utilitySmallModel": "oaicopilot/glm-5.1",
"chat.utilityModel": "oaicopilot/glm-5.1",
```

Value format is `vendor/id` with a SLASH — not a dot. The vendor is `oaicopilot` (the extension's registered vendor); the id is the model's `id` from `oaicopilot.models`. Using `oaicopilot.glm-5.1` (dot) **will not work** — it must be `oaicopilot/glm-5.1` (slash).

`glm-5.1` must be a model already listed in `oaicopilot.models` (it uses the global `oaicopilot.baseUrl` + generic key). It serves Copilot's background tasks (titles, summaries, follow-ups) — pick something fast and high-quota.

5.1 — Fallback option

If you don't have a separate fast model, you can tell Copilot to reuse your main agent model for utility tasks:

```
"chat.byokUtilityModelDefault": "mainAgent"
```

Enum: `"none"` (default — errors in BYOK), `"mainAgent"` (use the selected BYOK model), `"copilot"` (use GitHub's hosted utility models). A specific `chat.utilitySmallModel` / `chat.utilityModel` **takes precedence** over this.

6 Reload & use

1. **Reload VS Code** — Command Palette (`Ctrl+Shift+P`) → `Developer: Reload Window`. *Settings changes require a reload.*
2. Open GitHub Copilot Chat (`Ctrl+Shift+I`).
3. Click the **model picker** (model name in the chat header).
4. Select **DeepSeek-V3.2 (Foundry via LiteLLM)**.
5. Start chatting — the 429 and utility-model errors are gone.

Troubleshooting

Symptom	Cause	Fix
429 <code>RateLimitReached</code>	Burst exceeds Foundry quota	Confirm requests go through LiteLLM (check <code>baseUrl</code>). Lower <code>rpm</code> / <code>tpm</code> in the LiteLLM model group. Increase <code>num_retries</code> .
No utility model is configured for 'copilot-utility-small'	BYOK utility model not set, or wrong format	Set <code>chat.utilitySmallModel</code> with slash format <code>oaicopilot/glm-5.1</code> . Reload VS Code. Or set <code>chat.byokUtilityModelDefault: "mainAgent"</code> .
401 / <code>auth_error</code>	Bad LiteLLM key	Check the <code>headers.Authorization</code> Bearer value matches a valid LiteLLM master/virtual key.
404 <code>APIG.0101</code> or model not found	Model id mismatch	The extension <code>id</code> must exactly match the LiteLLM <code>model_name</code> .
Utility model still erroring after setting	Used dot instead of slash	Use <code>oaicopilot/glm-5.1</code> not <code>oaicopilot.glm-5.1</code> . Reload VS Code.
Prompted for "API key for litellm"	Per-provider key not in secret storage	Enter the LiteLLM key, or press Esc (custom header authenticates), or set via <code>OAI Copilot: Set OAI Compatible Multi-Provider API Key</code> .

Complete `settings.json` example

```
{
  "oaicopilot.baseUrl": "https://your-global-provider/v1",
  "oaicopilot.models": [
    {
      "id": "glm-5.1",
      "owned_by": "Huawei Cloud MaaS"
    },
    {
      "id": "deepseek-v3.2-foundry",
      "displayName": "DeepSeek-V3.2 (Foundry via LiteLLM)",
      "owned_by": "litellm",
      "baseUrl": "http://YOUR-SERVER:4000/v1",
      "apiMode": "openai",
      "context_length": 128000,
      "max_tokens": 32000,
      "headers": {
        "Authorization": "Bearer YOUR-LITELLM-KEY"
      }
    }
  ],
  "chat.utilitySmallModel": "oaicopilot/glm-5.1",
  "chat.utilityModel": "oaicopilot/glm-5.1"
}
```

Quick reference

What	Value / format
LiteLLM proxy URL	<code>http://YOUR-SERVER:4000/v1</code>
Foundry endpoint	<code>https://YOUR-RESOURCE.services.ai.azure.com/openai/v1</code>
Foundry deployment name	<code>DeepSeek-V3.2</code> (used as LiteLLM <code>model: openai/DeepSeek-V3.2</code>)
Extension vendor	<code>oaicopilot</code>
Utility model setting format	<code>vendor/id</code> → <code>oaicopilot/glm-5.1</code> (slash)
Extension id	<code>johnny-zhao.oai-compatible-copilot</code>
Key setting commands	<code>OAI Copilot: Set API Key</code> (generic) · <code>OAI Copilot: Set OAI Compatible Multi-Provider API Key</code> (per-provider)

Generated for the Azure AI Foundry + LiteLLM + VS Code Copilot (BYOK) deployment · Extension: oai-compatible-copilot (johnny-zhao) · LiteLLM v1.83.14